

NYXN —

PRUEBA TÉCNICA OFICIAL: FULL STACK DEVELOPER

Enfoque: Java, Spring Boot, Clean Architecture y Lógica Estructurada

Duración Máxima 2 Horas	Puntaje Total 100 Puntos	Mínimo para Avanzar 65 / 100 Puntos
-----------------------------------	------------------------------------	---

¡Hola! Nos alegra mucho que estés en esta etapa del proceso con **NYXN**. Esta prueba está diseñada para conocer tu criterio técnico más allá de la teoría. No buscamos respuestas perfectas; queremos ver cómo piensas, cómo tomas decisiones de arquitectura y cómo diseñas software moderno, robustas y de alta disponibilidad.

Nota importante: El uso de herramientas de IA (Claude, Copilot, Gemini) está completamente permitido y bienvenido en nuestro flujo de trabajo, siempre y cuando seas capaz de explicar y defender conceptualmente cada decisión tomada en la sesión posterior de revisión técnica.

SECCIÓN 1 — JAVA & SPRING BOOT (30 Puntos)

Punto 1.1 — API RESTful con Spring Boot

Construye una API RESTful con Spring Boot para gestionar un catálogo de productos de un e-commerce. La API debe exponer obligatoriamente los siguientes endpoints estructurados:

- GET /products - Listar todos los productos con paginación integrada.
- GET /products/{id} - Obtener el detalle de un producto específico por ID.
- POST /products - Crear un nuevo producto en el catálogo.
- PUT /products/{id} - Actualizar un producto existente.
- DELETE /products/{id} - Eliminar un producto del catálogo.

*El modelo de datos **Product** debe contener obligatoriamente los campos: id, name, description, price, stock, category y createdAt.*

Criterios de Evaluación:

- Estructura del proyecto organizada de forma limpia (Controller, Service, Repository).
- Uso semántico y correcto de anotaciones de Spring (@RestController, @Service, @Repository, @Transactional).
- Manejo global de excepciones mediante @ControllerAdvice devolviendo respuestas HTTP apropiadas (404, 400, 500).
- Paginación robusta implementada nativamente con Pageable de Spring Data.

- Validaciones estrictas de entrada usando Bean Validation (@NotNull, @Min, @Size, etc.).
- Documentación clara utilizando Swagger/OpenAPI anotando operaciones básicas (@Operation, @ApiResponse).

Punto 1.2 — Principios SOLID en Código Legacy

Analiza el siguiente fragmento de código Java e identifica qué principios SOLID viola de forma directa. Luego, refactorízalo aplicando principios de Clean Code y Clean Architecture:

```
public class OrderService {
    public void processOrder(int orderId) {
        Connection conn =
        DriverManager.getConnection("jdbc:postgresql://localhost/db", "user",
        "pass");
        PreparedStatement ps = conn.prepareStatement("SELECT * FROM orders
        WHERE id = " + orderId);
        ResultSet rs = ps.executeQuery();
        if (rs.next()) {
            double total = rs.getDouble("total");
            // Procesar pago
            HttpURLConnection payConn = (HttpURLConnection) new
            URL("https://payment.api/charge").openConnection();
            payConn.setRequestMethod("POST");
            // Enviar email
            Properties props = new Properties();
            props.put("mail.smtp.host", "smtp.gmail.com");
            Session session = Session.getInstance(props);
            // Actualizar inventario
            conn.prepareStatement("UPDATE inventory SET stock = stock - 1
            WHERE...").execute();
        }
    }
}
```

Criterios de Evaluación:

- Enumeración precisa de al menos 4 principios SOLID explícitamente violados.
- Refactorización desacoplada definiendo interfaces abstractas claras (PaymentService, EmailService, InventoryService).
- Inyección de dependencias estricta a través del constructor.
- Separación de responsabilidades de infraestructura (SRP aplicado correctamente).
- Uso propuesto o conceptual de ejecución asíncrona/no-bloqueante donde aplique.

SECCIÓN 2 — CLEAN ARCHITECTURE & ARQUITECTURA HEXAGONAL (25 Puntos)

Punto 2.1 — Diseño e Implementación de Arquitectura desacoplada

Diseña detalladamente la estructura de paquetes y capas lógicas para un módulo de gestión de pedidos en un esquema de Arquitectura Hexagonal (Ports & Adapters).

El módulo debe contemplar:

- Recibir un pedido desde una API REST (Adaptador Primario).
- Persistir la información en una base de datos PostgreSQL (Adaptador Secundario).
- Publicar un evento asíncrono a un tópico de Google Cloud Pub/Sub cuando el pedido se marque como 'Confirmado'.
- Enviar notificaciones de confirmación por email (Adaptador Secundario).

Responde de manera argumentada las siguientes preguntas clave:

- ¿Qué elementos y lógica de negocio residen en el Núcleo o Dominio?
- ¿Qué representa un Puerto en este ecosistema y cómo se diferencia de un Adaptador?
- ¿Dónde viven y cómo se ejecutan los Casos de Uso?
- Muestra gráficamente la estructura de paquetes bajo la ruta base `com.empresa.orders {...}` y define de forma explícita al menos 2 puertos de entrada y 2 puertos de salida.

Criterios de Evaluación:

- Estructura jerárquica de paquetes estrictamente coherente con los dominios de la Arquitectura Hexagonal.
- Definición técnica correcta de puertos (interfaces genéricas) de entrada y salida.
- Aislamiento total del Dominio: el core no debe depender de frameworks (Spring) ni de la infraestructura.
- Ubicación lógica de los Casos de Uso en la capa de Aplicación.
- Justificación técnica clara de por qué GCP Pub/Sub se categoriza como un adaptador secundario.

SECCIÓN 3 — LÓGICA DE DESARROLLO & ALGORITMOS (10 Puntos) [NUEVO]

Punto 3.1 — Control de Concurrencia e Integridad en Stock de Productos

Escenario Crítico: Durante un evento de alta demanda (Cyber-Day), múltiples transacciones concurrentes intentan comprar el mismo producto exactamente al mismo tiempo. Si dos hilos de ejecución leen el mismo stock disponible (ej. `stock = 1`) de forma simultánea, ambos podrían

procesar la venta, dejando el stock final de forma inconsistente o en valores negativos, provocando un problema crítico en la consistencia del negocio.

Plantea detalladamente tu estrategia técnica para mitigar este problema:

- Explica la diferencia entre bloqueo optimista (Optimistic Locking) y bloqueo pesimista (Pessimistic Locking). ¿Cuál elegirías para este escenario e-commerce transaccional de alta concurrencia y por qué?
- Muestra en pseudocódigo o código Java/Spring Boot cómo implementarías la solución elegida (puedes utilizar anotaciones de Spring Data JPA como `@Lock` o control de versiones con `@Version`).
- Menciona qué código de estado HTTP o manejo de excepciones retornarías al cliente cuando ocurra una colisión por concurrencia.

Criterios de Evaluación:

- Criterio y madurez senior para discernir entre estrategias de concurrencia (Optimista vs Pesimista) bajo picos de tráfico.
- Implementación limpia del algoritmo o código de bloqueo protegiendo la sección crítica.
- Estrategia clara de resiliencia o reintentos automáticos para no degradar la experiencia de usuario.

SECCIÓN 4 — BASES DE DATOS: POSTGRESQL, ORACLESQL Y REDIS (15 Puntos)

Punto 4.1 — SQL Avanzado Multi-Motor

Dado el siguiente modelo de datos entidad-relación estándar de un e-commerce:

- orders(id, customer_id, status, total, created_at)
- order_items(id, order_id, product_id, quantity, unit_price)
- products(id, name, category, stock)
- customers(id, name, email, country)

Escribe consultas SQL altamente optimizadas y compatibles tanto para PostgreSQL como para OracleSQL:

1. a) Top 5 clientes con mayor valor acumulado en compras en los últimos 30 días, incluyendo su volumen de órdenes y su ticket promedio por orden.
2. b) Productos con stock crítico (menos de 10 unidades) que registren ventas acumuladas superiores a 50 unidades dentro del último mes calendárico. Incluye nombre de producto y categoría.
3. c) Reporte consolidado de ventas diarias correspondientes a los últimos 7 días agrupado por categoría. Es imperativo incluir los días con ventas en cero (Usa LEFT JOIN y funciones generadoras de series como GENERATE_SERIES).

Criterios de Evaluación:

- Sintaxis correcta, óptima y directamente ejecutable.
- Uso estructurado de Expresiones de Tabla Comunes (CTEs) para maximizar la legibilidad y mantenimiento.
- Manejo exacto de funciones temporales en PostgreSQL (NOW(), INTERVAL) y OracleSQL (SYSDATE).
- Uso apropiado de funciones analíticas de ventana (RANK(), ROW_NUMBER()) según aplique.

Punto 4.2 — Redis como Capa de Caché Empresarial

Tu API de lectura de productos recibe ráfagas masivas de hasta 10,000 peticiones por minuto. Sin embargo, las propiedades de los productos (precios, descripciones) cambian como máximo una vez por hora.

Explica detalladamente y expón en código real de Spring Boot cómo implementarías una estrategia de caché distribuida usando abstracciones nativas (@Cacheable, @CacheEvict) o mediante clientes directos (Jedis/Lettuce). Responde: ¿Qué TTL configurarías y bajo qué lógica de negocio? ¿Cómo mitigarías el fenómeno de Cache Stampede?

SECCIÓN 5 — GOOGLE CLOUD PLATFORM & DOCKER (10 Puntos)

Punto 5.1 — Arquitectura de Despliegue Cloud Native e Inmutabilidad

Se requiere desplegar la aplicación Spring Boot sobre la nube de Google Cloud Platform (GCP). Responde con sólidos fundamentos técnicos a las siguientes pautas:

- ¿Qué servicio de cómputo elegirías para el despliegue de microservicios contenedores y por qué? (Justifica comparando Cloud Run, GKE o App Engine bajo los pilares de costo, escalabilidad automática y sobrecarga operacional).
- Escribe un Dockerfile optimizado para producción. Se requiere la implementación imperativa de Multi-Stage Builds para minimizar la superficie de ataque, reducir el peso de la imagen final y asegurar la ejecución bajo un usuario No-Root (seguridad reforzada).
- ¿Cómo centralizarías y consumirías los secretos, variables de entorno y llaves de APIs privadas en GCP? Explica la integración nativa con el servicio correspondiente.
- Diseña o describe de manera simplificada un pipeline de CI/CD automatizado mediante GitHub Actions o GCP Cloud Build para compilar, probar, empaquetar y desplegar de forma ininterrumpida al hacer merge a la rama principal (main).

SECCIÓN 6 — NODE.JS, NEXT.JS & INTELIGENCIA ARTIFICIAL (10 Puntos)

Punto 6.1 — Microservicio de Notificaciones y Agentes de IA Generativa

Parte A — Node.js & Patrones de Diseño: Escribe un microservicio robusto utilizando Node.js (NestJS o Express en TypeScript) que exponga un endpoint unificado de tipo POST /notify. Este endpoint procesará una carga útil con la estructura { userId, message, channel } y enrutará dinámicamente la notificación al proveedor destino ('email', 'sms', 'push'). Es obligatorio estructurar el código aplicando rigurosamente el patrón de diseño de software Strategy.

Parte B — Agente de Inteligencia Artificial (Claude): Describe de manera clara tu enfoque técnico para integrar a Claude (Anthropic API) como un agente autónomo e inteligente embebido dentro del flujo de gestión de órdenes de este e-commerce. Responde: ¿Qué tareas transaccionales delegarías directamente al LLM?, ¿Cómo asegurarías una comunicación segura bidireccional con tu backend?, ¿Optarías por un esquema de Tool Use (Function Calling) o por una arquitectura RAG? Argumenta.